DAY 2 — JAVA PROGRAMMING FUNDAMENTALS

Fundamental Topics Only • English Notes

Contents

1. Introduction to Java

2. Features of Java

3. Java Program Structure

4. JDK, JRE and JVM

5. Variables and Data Types

6. Type Casting

7. Operators

8. Conditional Statements

9. Loops

10. Input and Output

11. Basic Problem-Solving Concepts

12. Java Environment Setup

13. Quick Revision

1. Introduction to Java

Java is a high-level, class-based, object-oriented programming language. It is used to create desktop, web, mobile and enterprise applications.

Java source code is saved in a **.java** file. The Java compiler converts it into bytecode, usually stored in a **.class** file. The Java Virtual Machine (JVM) executes that bytecode on a compatible system.

2. Features of Java

Simple: Its syntax is structured and familiar to many programmers.

Object-oriented: Programs can be organized using classes and objects; Java supports encapsulation, inheritance and polymorphism.

Platform-independent: Bytecode can run on systems that provide a compatible JVM.

Secure: Java runtime mechanisms include bytecode verification and controlled memory access.

Robust: Exception handling and automatic memory management help make programs more reliable.

Multithreaded: Java supports running multiple threads within a program.

Portable: Java's standard behavior helps programs work consistently across supported platforms.

High performance: Just-in-time compilation can improve execution speed.

3. Basic Structure of a Java Program

A basic Java program commonly contains a class declaration and the `main()` method, where execution begins for a standard application.

`public static void main(String[] args)` is the standard entry-point signature. `System.out.println()` displays text and then moves to a new line.

4. JDK, JRE and JVM

JVM (Java Virtual Machine): Executes bytecode and provides runtime services such as memory management and garbage collection.

JRE (Java Runtime Environment): Traditionally describes the JVM plus libraries and other components required to run Java applications. Packaging varies in modern Java distributions.

JDK (Java Development Kit): Provides tools for developing Java applications, including the compiler `javac` and the Java launcher.

Component	Main purpose	Remember
JDK	Develop and compile programs	Used by developers
JRE	Provide runtime components	Used to run applications
JVM	Execute Java bytecode	Runs bytecode

5. Variables and Data Types

A **variable** is a named storage location used to hold a value. A variable has a data type, a name and, optionally, an initial value.

Primitive data types: byte, short, int, long, float, double, char and boolean.

Reference types: String, arrays and objects created from classes.

Declaration example: `int age = 20;` declares an integer variable named age and assigns it the value 20.

6. Type Casting

Type casting converts a value from one data type to another.

Widening conversion: Converts a type to a wider compatible type, such as int to double; it is usually automatic.

Narrowing conversion: Converts a wider type to a narrower type, such as double to int; it normally requires an explicit cast and may lose information.

7. Operators

An **operator** is a symbol that performs an operation on values or variables.

Arithmetic: +, -, *, /, % — calculations.

Relational: ==, !=, >, <, >=, <= — comparisons that produce a boolean result.

Logical: &&, ||, ! — combine or reverse boolean conditions.

Assignment: =, +=, -=, *=, /= — assign or update values.

Unary: ++, --, ! — operate on one operand.

Conditional operator: condition ? value1 : value2 — chooses one of two expressions.

Operator precedence: Multiplication and division generally occur before addition and subtraction. Parentheses can make the intended order clear.

8. Conditional Statements

Conditional statements allow a program to make decisions.

if: Executes a block when its condition is true.

if-else: Chooses between two alternatives.

else-if ladder: Checks multiple conditions in sequence.

Nested if: Places one if statement inside another.

switch: Selects a matching case from several alternatives. In traditional switch statements, `break` is commonly used to prevent execution from falling through to the next case.

9. Loops

Loops repeat a block of code. They are useful when an operation must be performed multiple times.

for loop: Often used when initialization, condition and update can be written together.

while loop: Checks the condition before each iteration. The body may execute zero times.

do-while loop: Checks the condition after the body, so the body executes at least once.

Loop control: `break` exits a loop; `continue` skips to the next iteration.

10. Input and Output

Output: `System.out.print()` displays text without automatically starting a new line; `System.out.println()` displays text and then starts a new line.

Input: The `Scanner` class can read keyboard input. Common methods include `nextInt()`, `nextDouble()`, `next()` and `nextLine()`. To use Scanner, import `java.util.Scanner`.

11. Basic Problem-Solving Concepts

Problem-solving in programming means understanding a task and planning a clear sequence of steps to solve it.

Algorithm: A finite, ordered set of steps used to solve a problem.

Pseudocode: An informal, language-independent way to describe an algorithm.

Flowchart: A diagram that represents steps and decisions using standard symbols.

Debugging: Finding and correcting errors in a program.

Testing: Running a program with selected inputs to check whether the results are correct.

12. Java Environment Setup

To compile and run a basic Java program, install a suitable JDK and use an editor or IDE.

Check installation: Run `java -version` and `javac -version` in a terminal or Command Prompt.

Compile: `javac Hello.java`

Run: `java Hello` (use the class name without the .java extension).

13. Quick Revision

Term	Meaning
JDK	Tools for Java development
JRE	Runtime components for Java applications
JVM	Executes bytecode
Variable	Named storage for a value
if-else	Decision-making
Loop	Repeats statements
Scanner	Reads input
Algorithm	Step-by-step solution
Debugging	Finding and fixing errors

Note: This version contains fundamental concepts and explanations only. The six coding exercises and their solutions have been excluded as requested.